DATASCIENCE, LEARNING AND APPLICATIONS

DALAS - Building a Dataset from a Warehouse

28 September 2026

Camelia Constantin

Machine Learning Training Dataset

SCIENCES SORBONNE UNIVERSITÉ

1. Goal : Churn Prediction

Predict whether a customer will stop purchasing within a forward window $[t_{\text{cutoff}}, t_{\text{cutoff}} + \Delta t]$.

2. The Observation Unit

One Row = One Observation defined by the pair (customer_id, t_{cutoff}).
A training sample is the state of a customer at a given **reference date** (t_{cutoff}).

- Customer 8842 at **1 Jan** : active history $\rightarrow$ Label = Active ($y = 0$).
- Customer 8842 at **1 June** : inactive history $\rightarrow$ Label = Churned ($y = 1$).

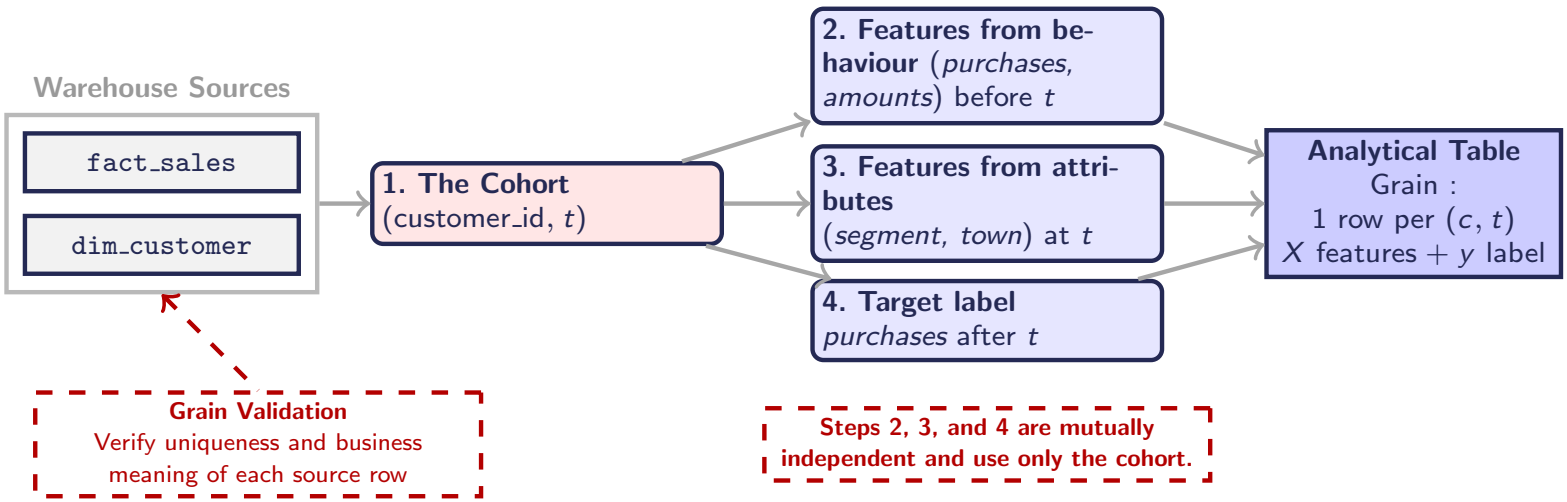
3. Table Structure to Produce

- **Features (X)** : Aggregations computed strictly on past events ($t < t_{\text{cutoff}}$) to prevent **data leakage**.
- **Target (y)** : Label computed on future events ($t \geq t_{\text{cutoff}}$).
- **Warehouse Storage** : Saved as a persistent SQL table to ensure exact dataset reproducibility for model training and auditing.

Dataset Construction Pipeline

Data Warehouse : Inherited Storage

Read-only access to a **pre-existing, centralised** system, **built for the analytical questions of the organisation**. The content of every column was already fixed by the analytics team.



Operations on the Warehouse Tables

1. The meaning of one row in each table. **2.** The preservation of past states. **3.** The computation of a feature relative to a date. **4.** The reasons a wrong figure for a feature could be undetected.

Tasks To Be Performed On the Warehouse

Analytical Task	Objective	Requirement
1. Exploration	Inspect schemas, grain, and temporal coverage	Documented column definitions
2. Cohort Selection	Delimit the target population and compute reference figures	A written eligibility rule
3. Dataset Building	Compute features (X) and label (y) at the chosen grain	Historised dimensions
4. Re-execution	Refresh the training data and compare successive runs	Reproducibility over time

Requirement of the Fourth Task

- **Stability** : an identical query on the same past window must return identical rows months later, **unless the period was reloaded**.
- **Traceability** : the comparison of two runs requires a record of the table computation and of the figures of this table.

The Warehouse as a Source of Data

Running Example : MegaStore

Context

A retail company with 200 shops, an e-commerce website, and a loyalty programme, operating **several separate systems**.

System	Recorded Events	Customer Identifier
Point of sale	In-shop purchases	Payment card hash, or none
E-commerce	Online orders and returns	`CUST-8842-B`
Loyalty	Customer profile and points	8842
Web server	Browsing logs	Browser cookie

Two Limitations for Data Science

Local identifiers : No foreign key links 8842 and `CUST-8842-B` to the same individual across systems.
Limited history : Operational systems purge past transactions when accounting periods close.

Analytical Objectives

(1) Global Reporting : Monthly revenue aggregated by region.
(2) Machine Learning : Predicting which customers are about to churn.
Questions that can't be answered from any single system alone.

OLTP vs. OLAP

SCIENCES SORBONNE UNIVERSITÉ

On the e-commerce database

```
SELECT status, ordered_on
FROM orders
WHERE order_id = 48213;
```

On the warehouse

```
SELECT region, month, SUM(amount)
FROM sales
GROUP BY region, month;
```

Dimension	OLTP (Transactional)	OLAP (Analytical)
Access pattern	Point lookups (1 row by key)	Large scans (millions of rows)
Columns accessed	Most columns per row	Few columns
Target latency	Milliseconds	Seconds to minutes
Optimal physical layout	Row-oriented storage	Columnar storage

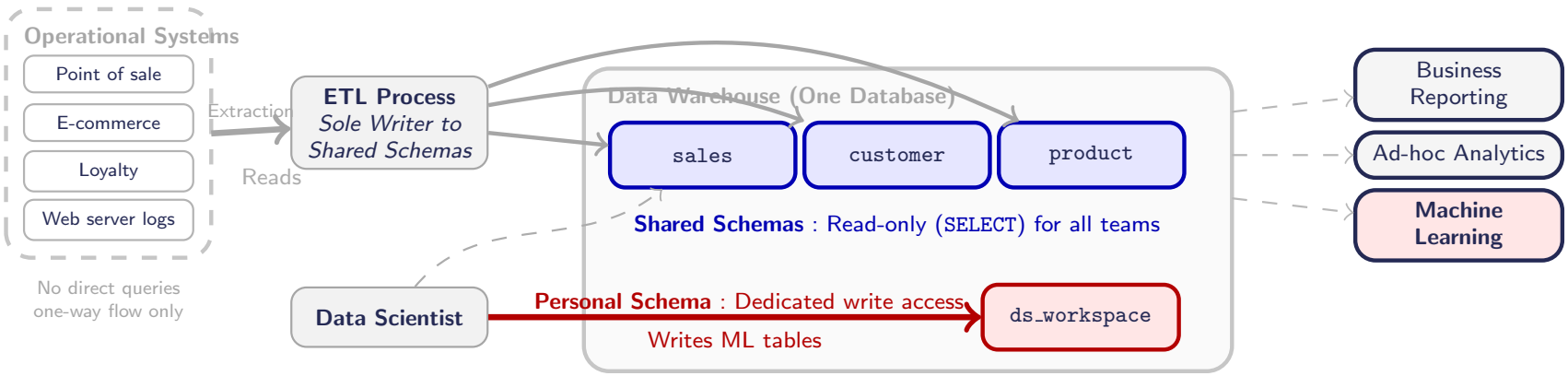
The Two Systems in Practice

- **Complementarity** : the analytical tables are computed from the operational databases, after the events have been recorded.
- **Workload separation** : an analytical scan run on an operational database locks tables and degrades the response times of the application.
- The queries and the transformations under study are written against the analytical system.

Database Organization : Schemas and the ETL Process

SCIENCES SORBONNE UNIVERSITÉ

Schema = logical namespace and container inside the database, grouping related tables and defining access permissions. Operational data moves into the warehouse only through an automated **ETL** process (*Extract*, *Transform*, *Load*). Analytical queries do not access operational databases directly.



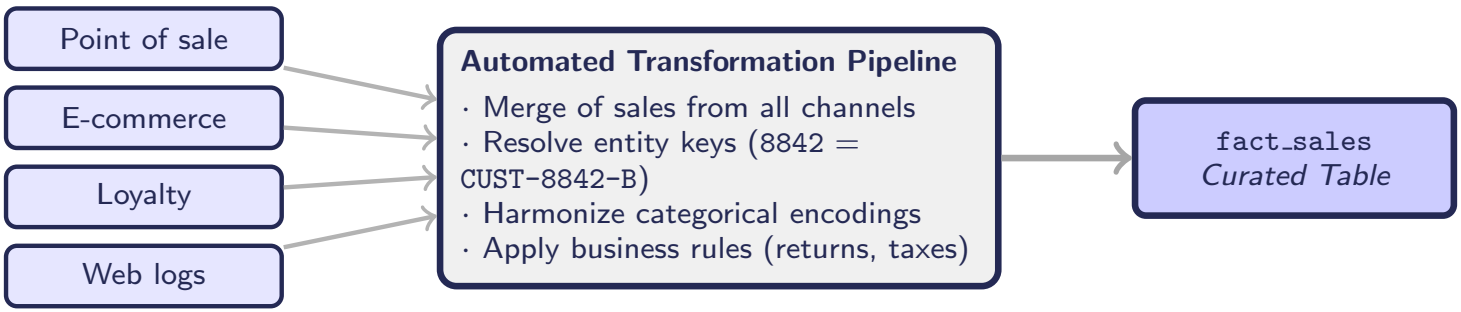
Direct Impact on Machine Learning

- **Production OLTP** : changes constantly with daily operations, preventing exact experiment replication.
- **Data Warehouse** : frozen between scheduled loads, guaranteeing reproducible training sets and verifiable feature values.

The Computation of a Warehouse Table

The Transformation Layer

Warehouse tables are not raw copies of operational tables. They are the output of **curated SQL transformation pipelines** re-executed during each scheduled batch load.



Automated execution on every ingestion cycle

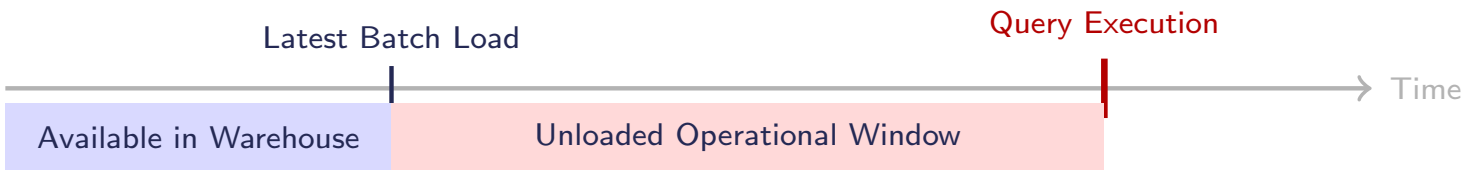
Advantage for Machine Learning

- **Feature Reliability** : Base tables incorporate validated enterprise definitions.
- **Efficiency** : Encoding harmonisation and identifier matching are performed by the load

Data Freshness and Latency

Ingestion Latency

A data warehouse reflects state of the operational systems up to its **latest completed batch load**. Operational events occurring after this load remain unloaded until the next cycle.



Offline Training Set Construction

Unaffected by latency. Historical observations are evaluated at past reference dates (t_{cutoff}) where all required history is available.

Online Model Inference

Constrained by latency. Real time predictions requiring immediate session events depend on operational data streams or caching, not on batch warehouse loads.

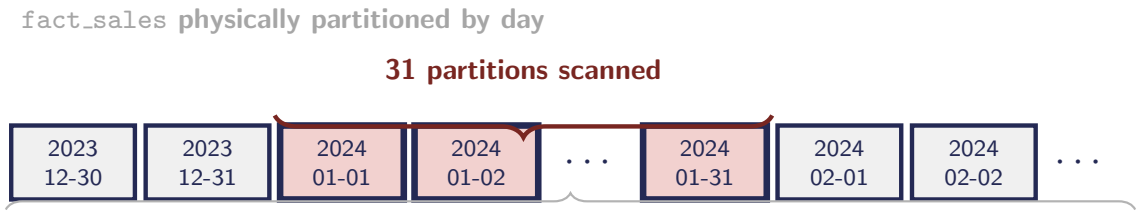
This Course

Offline analytical dataset construction and batch feature construction using SQL on historical warehouse tables.

Physical Storage : Partitioning and Partition Pruning

Physical Partitioning

Fact tables containing hundreds of millions of rows are physically divided into separate storage files (**partitions**), commonly split by transaction date.




```
/* Optimal: 31 partitions scanned (Pruning active) */
WHERE sale_date >= DATE '2024-01-01' AND sale_date < DATE '2024-02-01'

/* 1100 partitions read (Full scan forced) */
WHERE EXTRACT(YEAR FROM sale_date) = 2024
```

Partition Pruning Mechanism

- **Metadata Filtering** : The engine checks date boundaries for each partition file prior to reading data blocks.
- **I/O Reduction** : Filtering by date skips all irrelevant partitions automatically.
- **Impact on Feature Queries** : Correct date predicates avoid full table scans (**Two ways of writing the same question read 31 partitions, or 1100**).

The Effect of an UPDATE on the Previous Value

The Overwriting of Previous Values

Operational databases keep present transactions, UPDATE statements overwrite previous values in place, **permanently erasing past states**.



Retroactive Modification of a Reported Figure

- **A figure reported and later contradicted** : Past revenue and customer actions are retroactively reassigned to new attributes without any database warning.
- **A model fitted on values that no longer exist** : Re-extracting historical features after an update produces different numbers and causes lookahead leakage.

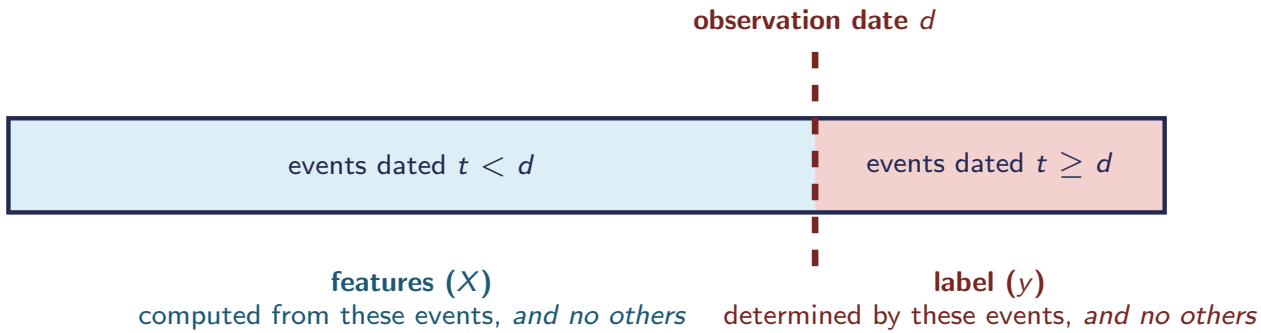
Warehouse Requirement

Analytical systems preserve immutable historical states over time (*historization mechanisms*).

Constraints on the Observation Date

Separation of the Two Windows

A model applied at date d answers on an outcome that has not occurred.
The features (X) are therefore computed from events dated $t < d$, and the label (y) from events dated $t \geq d$.



Consequences for the Extraction

- **Silent failure** : a column computed from events dated $t \geq d$ carries part of the answer $\Rightarrow$ a fitting library returns a high score.
- **Role of the warehouse** : the historised dimensions and the as-of join supply the state as it stood at d.

Warehouse Tables

Data Architecture : Three Possible Relational Settings

Data Warehouses are optimized for analytical queries.
Example : *Compute total revenue grouped by region and by month.*

Model	Structural Description	Joins	Architectural Trade-offs
A	**Normalized (3NF)** One table per entity, no redundancy : `sale_line` → `sale` → `shop` → `town` → `dept` → `region`	5	Join cost scales with the depth of the descriptive hierarchy
B	**One Big Table (OBT)** Single wide table with all attributes replicated per transaction row	0	Storage redundancy. Unsold products or inactive entities are not represented
C	**Dimensional (Star)** Numeric **facts** in a central table, **attributes** in dimension tables	2	Controlled redundancy by using dimensions. The number of joins depends on the dimensions.

This section : **Arrangement C (Dimensional Modeling)**.

Starting Point : Normalized Operational Schema (3NF)

SCIENCES SORBONNE UNIVERSITÉ

Third Normal Form (3NF)

Each entity occupies its own table with no descriptive redundancy. Attributes link to parent entities through explicit foreign keys.

Table	Attributes (Keys and Values)	Order of Magnitude
`sale_line`	`sale_id`, `product_id`, `quantity`, `amount`	5×10^8
`sale`	`sale_id`, `sale_date`, `shop_id`, `customer_id`	1.8×10^8
`shop`	`shop_id`, `name`, `surface`, `opening_date`, `town_id`	200
`town`	`town_id`, `name`, `dept_id`	≈ 150
`department`	`dept_id`, `name`, `region_id`	101 *(France)*
`region`	`region_id`, `name`	18 *(France)*

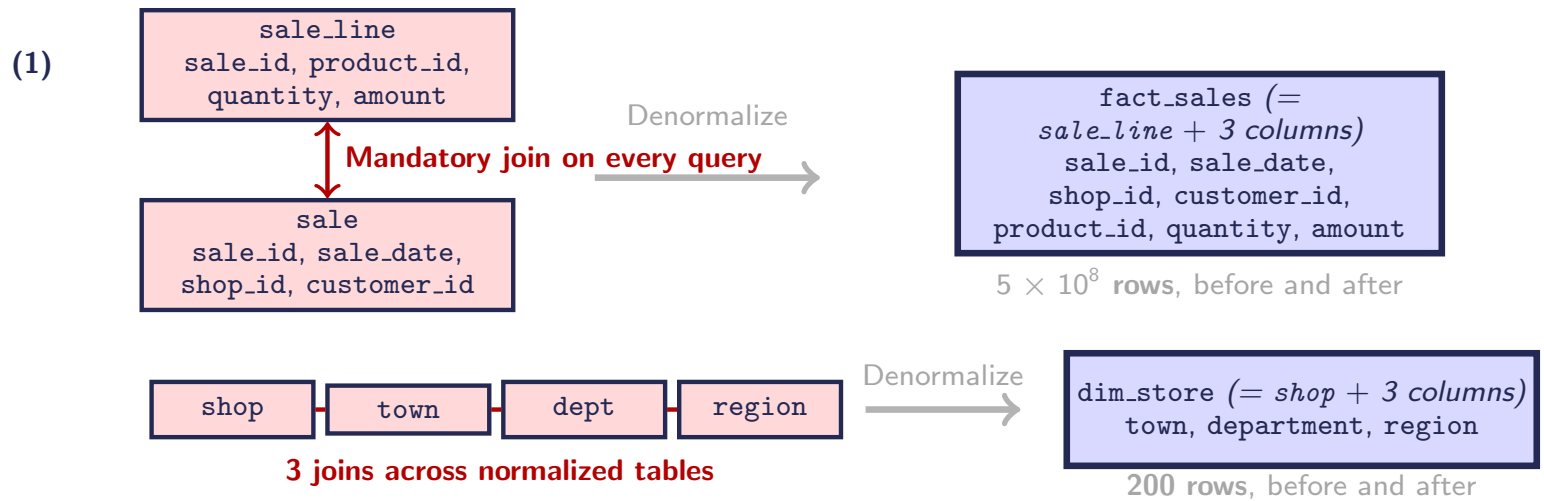
- **Join chain** : Aggregating by region requires traversing 5 relational joins :

$$\texttt{sale_line} \rightarrow \texttt{sale} \rightarrow \texttt{shop} \rightarrow \texttt{town} \rightarrow \texttt{department} \rightarrow \texttt{region}$$

- **Volume disparity** : 5×10^8 transaction lines are joined against tables of a few hundred rows. The cost of a query is determined by the reading of the transaction lines.

Two Denormalization Decisions

Reduce the number of joins from 5 to 1, in two steps : **(1)** merging transaction headers and lines, **(2)** flattening hierarchies.



Effect on the Query

- **Decision 1** : header and line merged ⇒ 1 join eliminated. `sale_id` is retained without a target table (**degenerate dimension**).
- **Decision 2** : a hierarchy is a sequence of attributes of decreasing detail (`shop` ⊂ `town` ⊂ `department` ⊂ `region`). Flattening it into `dim_store` ⇒ 3 joins eliminated.
- **Result** : 5 joins reduced to **1**, between `fact_sales` and `dim_store`.

Fact Tables

Facts

Observed event occurrence (a transaction line, a shipment, a click).

Three Categories of Columns

- **Measures** : Numeric values recorded for each event (`quantity`, `line_amount`) intended for aggregation (`SUM`, `AVG`).
- **Foreign Keys** : Relational identifiers pointing to contextual dimensions (`product_id`, `store_id`, `date_id`).
- **Degenerate Dimensions** : Transaction identifiers (`sale_id`) grouping related event lines without referencing an external table. Their purpose is to identify lines belonging to the same transaction.

- **Table shape** : Many rows, few columns (5×10^8 rows, approximately 10 columns).
- **Columnar Storage Layout** : Column are stored contiguously, analytical aggregations scan only 2 or 3 columns across millions of rows.

Example of a Fact Table : sales.fact_sales

sales.fact_sales (5×10^8 rows)							
sale_id	sale_date	date_id	store_id	product_id	customer_key	quantity	line_amount
90114	2024-01-15	20240115	12	4471	55231	2	39.80
90114	2024-01-15	20240115	12	1093	55231	1	12.50
90115	2024-01-15	20240115	07	4471	21094	1	19.90
90233	2024-01-16	20240116	12	8802	60417	3	27.00

Structural Characteristics

- **Measures and Keys Only** : Contains all numerical records, no textual attributes (no region, category, or month name).
- **Scan Dominated Cost** : Query runtime is determined almost entirely by the fact table scan (joins with small dimensions add minimal overhead).

Grain and Keys

- **Line item grain** : One row represents one line item (Rows 1 and 2 belong to the same basket via the degenerate dimension `sale_id`).
- **Partition key (`sale_date`)** : Raw temporal field used in `WHERE` clauses for partition pruning.
- **Dimension key (`date_id`)** : Foreign key referencing the calendar dimension for aggregations by month or quarter.

Dimension Tables

Dimension

Dimension Table : stores descriptive context for entities referenced by event measurements (stores, products, customers, calendar dates). Fact tables link to dimensions using foreign keys.

Table Shape : Few Rows, Many Columns

- **One row per entity** : Represents one described entity.
- **Many descriptive columns** : Stores all contextual attributes used for filtering (`WHERE`) and aggregation (`GROUP BY`).
- **Example (`dim_store`)** : 200 rows, 8 columns containing surface area, town, department, and region.

Distinguishing Entity from Event

- **Row semantics** : Describes an **entity**, not an operational **event**.
- **Different Cardinalities** : `dim_store` has 200 rows and `dim_customer` has 4×10^6 rows. Both are dimensions.
- **Controlled redundancy** : Flattening repeats each region name across its $\sim$20 stores in `dim_store`. Each batch load rewrites the table entirely, so the copies stay consistent.

Reading a Fact Row Through Its Dimensions

SCIENCES SORBONNE UNIVERSITÉ

product.dim_product			
product_id	label	sub_category	category
4471	Olive oil 1L	Oils	Grocery
1093	Ground coffee	Hot drinks	Grocery

sales.dim_store			
store_id	town	department	region
12	Toulouse	Haute-Garonne	Occitanie
07	Montpellier	Herault	Occitanie

Decoding the First Fact Row

Combining the first row of `fact_sales` with both dimension tables resolves the full transaction event :

On 15 January 2024, in Toulouse (Occitanie), customer 55231 purchased 2 bottles of olive oil (Grocery) for 39.80.

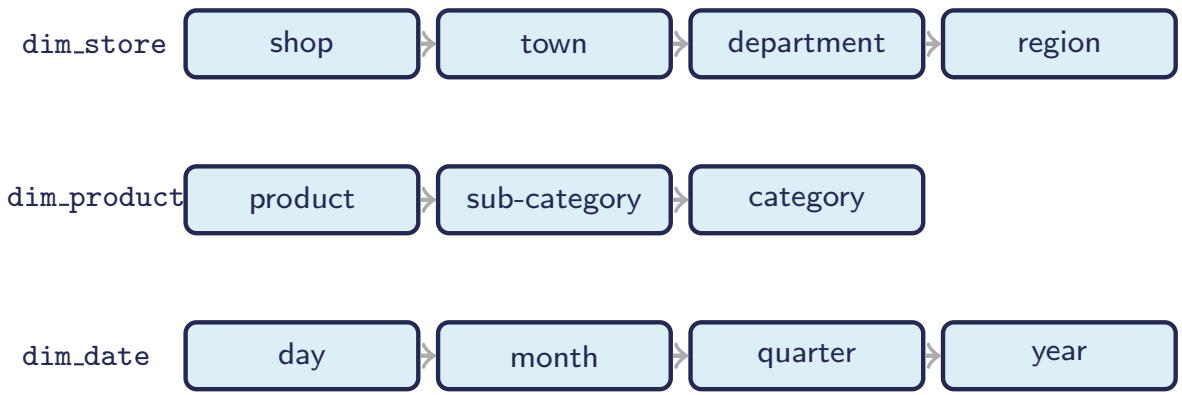
Role in the Construction of Features

- **Descriptive context** : Fact rows store numeric identifiers, while dimensions supply categorical hierarchies.
- **Aggregation axes** : Dimension columns define the grouping keys used to compute behavioral features (e.g. basket share in `Grocery`, preferred store `town`).

Hierarchies Inside Dimension Tables

Hierarchies Across the Three Dimensions

The flattening applied to `dim_store` applies to every dimension : each level of a hierarchy becomes a column of the table.



Grouping Without Extra Joins

- **Direct aggregation** : Grouping by any level requires selecting that column in the `GROUP BY` clause without additional joins.
- **Construction of features** : A feature can be computed at any level, such as category level spending and department level trends.

Surrogate Keys vs. Business Keys

SCIENCES SORBONNE UNIVERSITÉ

customer.dim_customer			
customer_key	customer_id	city	segment
55231	8842	Lyon	standard
55884	8842	Paris	premium
21094	3310	Toulouse	standard

Two Categories of Identifiers

- **Surrogate Key (`customer_key`)** : Synthetic integer generated during warehouse ingestion. Uniquely identifies a single **state** of the entity.
- **Business / Natural Key (`customer_id`)** : Identifier originating from operational systems (CRM, loyalty). Identifies the **real-world entity** across time.

Join Integrity in the Constructions of Features

- **Fact-to-dimension links** : Fact tables store `customer_key`, which designates the customer state recorded at the moment of the purchase.
- **Fan-out prevention** : Joining on `customer_id` matches multiple historical rows, silently duplicating transaction amounts and corrupting aggregated features.

Identifying Table Types in Unfamiliar Schemas

SCIENCES SORBONNE UNIVERSITÉ

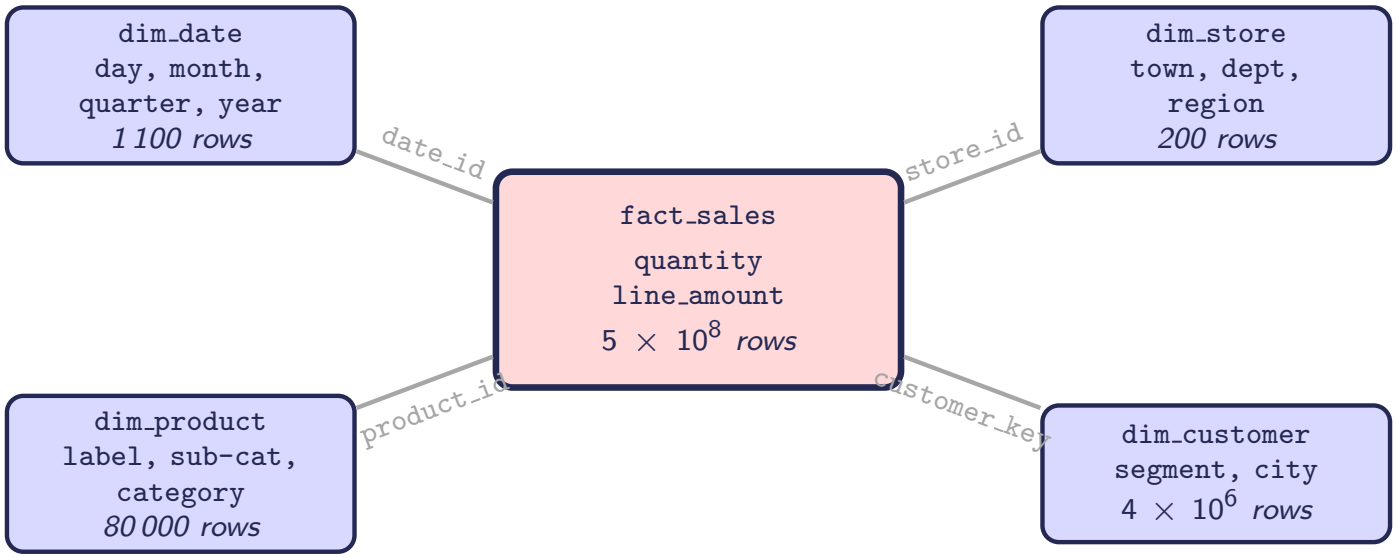
Classification Heuristics

In enterprise warehouses, shared schemas consist primarily of fact and dimension tables. Properties used to determine the nature of an unknown table :

Property	Fact Table	Dimension Table
Row Count (`COUNT(*)`)	Huge : 10^8 to 10^9+ rows	Usually small : 10^2 to 10^4 (*Exception : customer/user dim* $\approx 10^6$–10^7)
Dominant Types	Mostly integers (keys) and floats (measures)	Mostly strings, categoricals, and flags
Shape	Many rows, few columns	Fewer rows, many columns

- **Prefixes are optional** : The prefixes `fact_` and `dim_` are best practices, but production tables often lack them.
- **How to decide** : When the two properties above disagree, the nature of the table is settled by what one row represents : an **event** that happened (Fact), or an **entity** that is described (Dimension).

Reference Architecture : The MegaStore Star Schema



Properties

- **Isolated Dimensions** : Dimensions never join to each other directly (any inter-entity relationship must traverse `fact_sales`).
- **Query pattern** : Analytical extractions start at the central fact table and attach only the necessary dimensions.
- **Execution** : The volume disparity (5×10^8 rows for fact tables, a few hundred for dimensions) allows the engine to hold the dimensions in memory during the join.

This architecture is implemented in PostgreSQL, Snowflake, Google BigQuery, Amazon Redshift, Databricks, ClickHouse, DuckDB, etc.

Date Dimension

Limitations of Native Date Types

A scalar `DATE` represents a point in time, but cannot provide business, operational, or regional calendar attributes without external metadata.

Analytical Attribute	**Why `DATE` Alone Is Insufficient**
Public Holiday	Depends on jurisdiction and regional calendar
Store Trading Day	Depends on company operational policy and local bylaws
Promotional Period	Defined by the commercial calendar of the company
Fiscal Quarter / Year	Organization-specific accounting calendar (e.g., starting in April)

Implementation and Schema Properties

- **Precalculated metadata** : Calendar attributes are stored as static columns rather than evaluated at each query.
- **Table grain and size** : `dim_date`, exactly one row per calendar day ($\approx 1\,100$ rows for 3 years).
- **Key conventions** : Integer surrogate key in the form 20240115 in existing warehouses, or a `DATE` column in recent ones.

Table Grain

SCIENCES SORBONNE UNIVERSITÉ

Grain : meaning of one row.
Example for `fact_sales` : *One product item within a single purchase transaction* (the two rows sharing `sale_id` 90114 belong to one and the same sale).

Candidate Grain	Row Count	Limitations
One line item per sale	5.0×10^8	None (all details preserved)
One sale (basket level)	1.8×10^8	Cannot analyze product co-occurrence
One store per day	$\approx 2.2 \times 10^5$	Eliminates all product and customer analysis

Grain Selection

- **Irreversible** : fine-grained data can be rolled up via `GROUP BY`, can't recover lost granularity.
- **Consequence** : the grain received determines which features can be computed.

Wrong Grain

Target Metric

Compute the average basket amount (average total spend per purchase transaction).

```sql
/* INCORRECT: Average item amount */
SELECT AVG(line_amount)
FROM sales.fact_sales;  -- Result: 30
```

```sql
/* CORRECT: Average basket amount */
SELECT AVG(basket_amount)
FROM (
  SELECT sale_id,
         SUM(line_amount) AS
     basket_amount
  FROM sales.fact_sales
  GROUP BY sale_id); -- Result: 50
```

sale_id	line_amount	Basket Total
90114	20	
90114	15	
90114	25	**60**
90115	40	**40**
90233	50	**50**

$$\text{Avg Line} = \frac{150}{5} = 30$$

$$\text{Avg Basket} = \frac{150}{3} = 50$$

- **Misleading naming** : The name `fact_sales` **suggests one row per sale**.
- **Sampling limits** : A query with `LIMIT 100` on 5×10^8 rows rarely displays duplicate `sale_id` values together.
- **Plausible results** : Both 30 and 50 seem realistic.

Empirical Verification of Table Grain

Two Steps

1. **Formulate candidate key** : Identify columns that should uniquely distinguish records.
2. **Compare cardinalities** : Check whether `COUNT(*)` matches the distinct count of candidate keys.

```
SELECT (SELECT COUNT(*) FROM sales.fact_sales)   AS n_rows,
  (SELECT COUNT(DISTINCT sale_id) FROM sales.fact_sales)   AS proposal_1,
  (SELECT COUNT(*) FROM (SELECT DISTINCT sale_id, product_id
                         FROM sales.fact_sales) k) AS proposal_2;
--  n_rows = 502 118 440 | proposal_1 = 179 328 014 | proposal_2 = 492 273 961
```

Interpretation of Cardinalities

- **Proposal 1 (`sale_id`)** : Refuted by factor 2.8. Confirms that rows represent line items, not full sales.
- **Proposal 2 (`sale_id, product_id`)** : Refuted by 2% ($4.92 \times 10^8 \neq 5.02 \times 10^8$). The same product can appear on multiple lines in one sale under different pricing or promotion conditions.
- **Consequence** : A 2% gap looks negligible, but every aggregation on it is wrong. True grain : one row per distinct price applied to a product within a sale.

Unresolved Foreign Keys

Unresolved key (e.g. new store) : rejecting the row corrupts sums, `NULL` drops it on inner joins. Warehouse convention : remap to a dedicated row, key `-1`, labeled `'Unknown'`.

sale_id	store_id	amount
90233	**88 → -1**	27.00

store_id	town	region
-1	**Unknown**	**Unknown**

```sql
-- 1. convention used by this warehouse
SELECT * FROM sales.dim_store WHERE store_id <= 0; -- -1 | Unknown

-- 2. proportion affected in this extraction
SELECT d.region, SUM(f.line_amount) FROM sales.fact_sales f
JOIN sales.dim_store d ON d.store_id = f.store_id
WHERE f.sale_date >= '2024-01-01' AND f.sale_date < '2025-01-01'
GROUP BY d.region; -- Unknown: 1% of revenue
```

Two Decisions

- **Exclusion** : totals no longer reconcile with official reports.
- **Retention** : model may learn load failures as valid categories.

A high proportion of unresolved keys = an upstream failure and is reported to the analytics team. A dimension loaded after its facts gives the same result : a *late-arriving dimension*.

SQL Three-Valued Logic

Three-Valued Logic (3VL) in Predicate Evaluation

In SQL, comparisons involving `NULL` evaluate to `UNKNOWN` rather than `TRUE` or `FALSE`. A `WHERE` clause filters out all records where the condition evaluates to `FALSE` or `UNKNOWN`.

```
/* Silently drops all rows where status IS NULL */
SELECT COUNT(*) FROM sales.fact_sales
WHERE status <> 'cancelled';

/* CORRECT: Preserves NULL values */
WHERE status IS DISTINCT FROM 'cancelled'        -- Standard SQL:1999
WHERE status <> 'cancelled' OR status IS NULL    -- portable equivalent
```

Impact on Fact Tables

- **Default state semantics** : In many operational schemas, a `NULL` status designates normal, unflagged transactions (e.g., successful deliveries without incidents).
- **Data exclusion** : An inequality filter such as `status <> 'cancelled'` removes every row whose status is not recorded, without any error or warning.
- **Preliminary inspection** : the distribution of `NULL` values is examined before any filter is written on a status column.

Schema Exploration and Metadata Profiling

SCIENCES SORBONNE UNIVERSITÉ

```sql
/* 1. Discover tables within schema */
SELECT table_schema, table_name FROM information_schema.tables
WHERE table_name LIKE '%sale%';

/* 2. Inspect attribute definitions and data types */
SELECT column_name, data_type FROM information_schema.columns
WHERE table_schema = 'sales' AND table_name = 'fact_sales';

SELECT * FROM sales.fact_sales LIMIT 100; /* 3. Sample records */
SELECT COUNT(*) FROM sales.fact_sales; /* 4. Determine table volume */
```

Data Type Difference : `DECIMAL` vs. Client-Side `float64`

- **Exact SQL storage** : Monetary values use fixed-point `DECIMAL`, avoiding the representation errors of binary floating point.
- **Driver casting** : Python drivers cast `DECIMAL` either to `decimal.Decimal` (`object` dtype) or directly to IEEE 754 `float64`.
- **Divergence between the two totals** : a total aggregated in SQL and the same total aggregated in pandas may differ on the trailing decimals. The dtypes obtained are inspected after each extraction.

Catalog Portability Across Engines

Standard ANSI : `information_schema` in PostgreSQL, Snowflake, and MySQL.
Engine variations : BigQuery exposes it per dataset, SQLite relies on `PRAGMA table_info(table_name)`.

Temporal Coverage and Volume Continuity

```sql
/* 1. does the required history exist? the task needs 455 days */
SELECT MIN(sale_date), MAX(sale_date), COUNT(*) FROM sales.fact_sales;

/* 2. Profile monthly transaction distribution to detect breaks */
SELECT EXTRACT(YEAR FROM sale_date) AS yr, EXTRACT(MONTH FROM sale_date) AS mo, COUNT(*) AS
      monthly_rows
FROM sales.fact_sales
GROUP BY yr, mo ORDER BY yr, mo;
```

2024-02	2024-03	2024-04	2024-05	2024-06
41.2 M	40.8 M	**9.1 M**	**52.7 M**	**53.1 M**

April : a single month below the others, which returns to its level afterwards.

From May : a new level, which persists.

Causes of Structural Breaks

- **Ingestion failure** : Incomplete pipeline runs or missing partitions (e.g. April drop).
- **Source migration** : Schema modifications or the replacement of an operational database.
- **Perimeter expansion** : Addition of new sales channels or acquired store networks (e.g. May baseline shift).

Consequence for the Training Data

A training set spanning a structural break is built on two different populations (the model learns the break, not the behaviour).

Join and GROUP BY

WHERE and HAVING

Two Distinct Queries

Total spend of customer 8842 in 2024, and *total spend of the customers active in 2024*. (Customer history : two purchases in 2024 (30, 50) and two in 2023 (70, 50)).

Spend in 2024 : WHERE

```
SELECT customer_id,
       SUM(line_amount) AS spend_2024
FROM sales.fact_sales
WHERE sale_date >= DATE '2024-01-01' AND
      sale_date < DATE '2025-01-01'
GROUP BY customer_id;
```

Rows of 2023 discarded first → **80**

Spend of Customers Active in 2024 : HAVING

```
SELECT customer_id,
       SUM(line_amount) AS total_spend
FROM sales.fact_sales
GROUP BY customer_id
HAVING MAX(sale_date) >= DATE '2024-01-01';
```

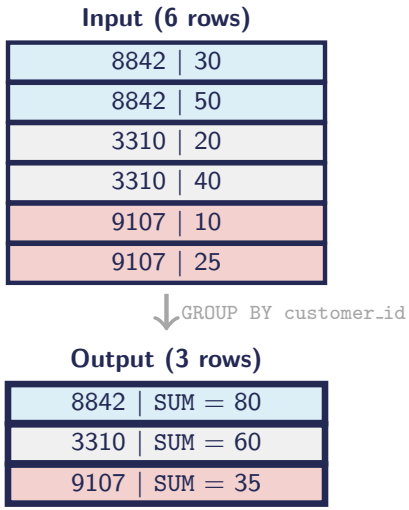
All four rows aggregated → **200**

Condition Target	SQL Clause
Individual column value	WHERE (evaluated before GROUP BY)
Aggregate expression	HAVING (evaluated after GROUP BY)

- **Syntax rule** : `WHERE SUM(line_amount) > 1000` fails because groups do not exist at row-filtering stage.
- **I/O optimization** : `WHERE` enables partition pruning and index filtering. `HAVING` requires scanning and grouping all rows first.

GROUP BY : Change of Grain

Input : Table/relation grain ⟶ Output : **Grain defined strictly by the grouping keys.**



Selected Column	Engine Behavior
In `GROUP BY`	Preserved as group key
Inside aggregate	Collapsed to 1 scalar per group
Neither	**Syntax error** in standard SQL (PostgreSQL, Snowflake). *Arbitrary scalar* in SQLite / non-strict MySQL.

Row Collapsing vs. Windowing

- **Loss of row detail** : Raw records cannot be referenced after aggregation.
- **Preserving row grain** : To compute group metrics without collapsing rows, use window functions (`OVER (PARTITION BY ...)`).

Date Dimensions : Partition Pruning vs. Grouping

SCIENCES SORBONNE UNIVERSITÉ

Analytical Query

Compute total revenue by region and month for year 2024.

fact_sales			
sale_date	date_id	store_id	line_amount
2024-01-15	20240115	12	39.80

dim_store		
store_id	town	region
12	Toulouse	Occitanie

dim_date		
date_id	month	is_holiday
20240115	2024-01	false

```
SELECT st.region, dt.month, SUM(f.line_amount) AS revenue
FROM sales.fact_sales f
JOIN sales.dim_store st ON st.store_id = f.store_id
JOIN sales.dim_date  dt ON dt.date_id = f.date_id
WHERE f.sale_date >= DATE '2024-01-01' AND f.sale_date <  DATE '2025-01-01'
GROUP BY st.region, dt.month;
```

Roles of Date Columns

- `WHERE f.sale_date` : partitioning column of `fact_sales`, triggers **partition pruning**.
- `GROUP BY dt.month` : `dim_date` supplies the month.
- **The variant to avoid** : Same result for `WHERE dt.year = 2024`, **every partition** is read. Finding the partitions of `fact_sales` that hold rows pointing to a 2024 row in `dim_date` requires opening these partitions to perform the join.

Data Validation

customer_id	coupon
8842	A1
8842	A1
8842	*NULL*
3310	B7
3310	B7
9107	C2

Expression	Result	Semantics
`COUNT(*)`	6	All rows
`COUNT(coupon)`	5	Non-NULLs
`COUNT(DISTINCT coupon)`	3	Unique non-NULLs

Diagnostic 1 : Measuring Missing Values

`COUNT(*) - COUNT(col)` yields the exact number of `NULL` entries.

- **Base tables** : the difference gives the number of values not recorded.
- **Result of an outer join** : the difference counts both the values not recorded and the rows without a match.

Diagnostic 2 : Checking Table Grain and Join Fan-Out

```
SELECT COUNT(*) AS total_rows, COUNT(DISTINCT customer_id) AS distinct_entities
FROM table_or_query_result;
```

- **total_rows** = **distinct_entities** : Exactly one record per entity.
- **total_rows** > **distinct_entities** : on a table, the grain is finer than one row per entity and on the result of a join or a join has multiplied the rows (join fan-out).

WITH (Common Table Expression) : Query Structure

SCIENCES SORBONNE UNIVERSITÉ

Subquery Limitations

Nested queries are read inside-out and cannot be isolated. When an output is incorrect, intermediate states cannot be inspected directly.

Nested Subquery (Inside-Out)

```
SELECT AVG(total_spend)
FROM (SELECT customer_id,
       SUM(line_amount) AS total_spend
  FROM sales.fact_sales GROUP BY customer_id
) sub;
```

Named CTE (Top-to-Bottom)

```
WITH per_customer AS (SELECT customer_id,
            SUM(line_amount) AS total_spend
  FROM sales.fact_sales GROUP BY customer_id
)
SELECT AVG(total_spend) FROM per_customer;
```

Advantages of Common Table Expressions (CTE)

- **Pipeline readability** : Code executes in logical declaration order.
- **Explicit grain declaration** : A descriptive CTE name (per_customer) documents the expected record grain.
- **Inspection of a stage** : the final SELECT can be replaced by a query on any named stage.

```
/* Inspecting intermediate CTE output during query development */
WITH per_customer AS ( ... )
SELECT COUNT(*) AS total_rows, COUNT(DISTINCT customer_id) AS entities FROM per_customer;
-- different counts: the grain is finer.
```

Silent Loss 1 : INNER vs. LEFT JOIN

SCIENCES SORBONNE UNIVERSITÉ

INNER JOIN : Keeps only matched rows, entities without transactions are dropped.
LEFT JOIN : Preserves all entities from the left table, missing columns are set to NULL.

```sql
/* 1. INNER JOIN: Drops inactive customers */
SELECT COUNT(DISTINCT c.customer_id)
FROM customer.dim_customer c JOIN sales.fact_sales o USING(customer_key);       -- 3,800,000

/* 2. LEFT JOIN: Preserves complete customer base */
SELECT COUNT(DISTINCT c.customer_id)
FROM customer.dim_customer c LEFT JOIN sales.fact_sales o USING(customer_key);  -- 4,000,000
```

Impact on Churn

The 200 000 missing customers have no row in fact_sales.
For a **churn prediction model**, these inactive users represent the core target population. An INNER JOIN discards them completely.

Analytical Scope	Join Type
Cohort of active buyers only	INNER JOIN
Entire customer base (every analysis in which the absence of a purchase is informative)	LEFT JOIN (with COALESCE)

Silent Loss 2 : NULL in Aggregates

Every aggregate except `COUNT(*)` **ignores** `NULL` : it is removed from the computation, **not** treated as zero.

cust.	line_amount
8842	30
8842	*NULL*
3310	50
9107	*NULL*

Rows exist
the amount is not recorded.

On These Four Rows

`COUNT(*)` = 4 `COUNT(line_amount)` = 2 `SUM` = 80
`AVG(line_amount)` = **40**, not 20 (and not marked as partial)
For customer 9107 alone : `SUM` is `NULL`, not 0

```
SELECT c.customer_id, COALESCE(SUM(f.line_amount), 0) AS total
FROM customer.dim_customer c
LEFT JOIN sales.fact_sales f ON f.customer_key = c.customer_key
GROUP BY c.customer_id;
```

Semantic Distinction : Zero vs. Missing

- **No row at all** : No fact row means `SUM` returns `NULL` though spend is zero (use `COALESCE(..., 0)`).
- **A recorded row with a missing measure** : for a continuous quantity (sensor reading, rating), `NULL` means *unknown* (replacing it with 0 introduces an unobserved value).

SQL or pandas

SCIENCES SORBONNE UNIVERSITÉ

Choosing SQL Over pandas for Transformations.

Metric	Raw Fact Table	Aggregated Target
Row count	5×10^8 rows	$\sim 4 \times 10^5$ rows
Memory footprint	∼50 GB	Tens of MBs
Fits in the memory of the workstation	**No**	**Yes**

Server-Side Computation and Network Transfer

- **Compute next to storage** : The SQL engine filters and aggregates partitions where the data is stored.
- **Volume transferred** : only the resulting ∼40 MB crosses the network, instead of 50 GB.

SQL and pandas

- Use **SQL** for volume reduction (filtering, joining, grouping). Transition to **pandas** once the table fits in memory.
- `pd.merge()` defaults to `how='inner'`. Omitting the argument silently drops unmatched rows, like an inner join.

Validity of Computed Metrics

Three Forms of Silent Error

Errors Without a Message

A query can execute without error and return a plausible figure that does not answer the question asked. No warning is issued by the engine.

Failure Form	Root Cause & Mechanism
Measure additivity	Column that cannot be summed along every dimension (stocks along time, rates, averages)
Join fan-out	A 1-to-many join duplicates rows before aggregation, silently inflating sums and counts.
Join truncation	An `INNER JOIN` discards unmatched entities, computing metrics on an incomplete cohort.

- **Independence of the three modes** : the three can occur in the same query, and each one leaves the others undetected.
- **One check per mode** : each of the three requires its own verification before the aggregate is used.

Form 1 : Measure Additivity

Measure Type	Summable Dimensions	Examples
Additive	All dimensions	Revenue, quantity sold, event count
Semi-additive	All dimensions **except time**	Inventory stock, bank balance, active headcounts
Non-additive	**None** (non-summable)	Profit margin %, unit price, conversion rate, pre-computed averages

Semi-Additive Measures

- **Spatial sum (valid)** : Adding warehouse stock across 10 locations on the same day gives the total current stock.
- **Temporal sum (invalid)** : Summing one warehouse's daily stock over 30 days yields $\approx 30\times$ the actual inventory. The value of the last day of the period, or the average over the period, is used instead.

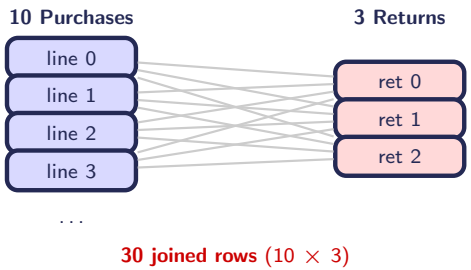
SQL Execution

- **The type carries no indication of additivity** : measures are stored as ordinary `NUMERIC/FLOAT`, with no indication of whether summing is valid.
- **Silent aggregation** : `SUM()` runs on semi-additive or non-additive columns without error, so the additivity must be established before the aggregate is written.

Form 2 : Join Fan-Out (Row Multiplication)

Case Study : Joining Two Fact Tables on Entity Key

Compute total purchases and returns for customer 8842 in 2024.



30 joined rows (10×3)

Metric	True	Computed	Distortion
Purchased amount	400	**1 200**	$\times 3$
Returned amount	60	**600**	$\times 10$

```sql
/*  Joining two 1-to-many fact tables directly */
SELECT d.customer_id,
       SUM(s.line_amount)   AS total_purchased,
       SUM(r.return_amount) AS total_returned
FROM sales.fact_sales s
JOIN sales.fact_returns r  USING (customer_key)
JOIN customer.dim_customer d USING (customer_key)
WHERE s.sale_date   >= DATE '2024-01-01' AND s.sale_date   < DATE '2025-01-01'
  AND r.return_date >= DATE '2024-01-01' AND r.return_date < DATE '2025-01-01'
GROUP BY d.customer_id;
```

The Error is Unnoticed

- The sums are distorted by different factors ($3\times$ and $10\times$), making later correction impossible.
- An annual total of 1 200 spent and 600 returned looks like realistic customer activity, passes inspection.

Solution : Aggregate to Target Grain Before Joining

SCIENCES SORBONNE UNIVERSITÉ

The Solution

Each fact table is aggregated to one row per customer before the join, so that no row is multiplied.

```
WITH purchased AS (
  /* Stage 1: Exactly 1 row per purchasing customer */
  SELECT d.customer_id, SUM(f.line_amount) AS amount_purchased
  FROM sales.fact_sales f
  JOIN customer.dim_customer d USING (customer_key)
  WHERE f.sale_date >= DATE '2024-01-01' AND f.sale_date < DATE '2025-01-01'
  GROUP BY d.customer_id
),
returned AS (
  /* Stage 2: Exactly 1 row per returning customer */
  SELECT d.customer_id, SUM(f.return_amount) AS amount_returned
  FROM sales.fact_returns f
  JOIN customer.dim_customer d USING (customer_key)
  WHERE f.sale_date >= DATE '2024-01-01' AND f.sale_date < DATE '2025-01-01'
  GROUP BY customer_id
)
/* Stage 3: 1-to-1 join on aligned grain */
SELECT customer_id, p.amount_purchased, COALESCE(r.amount_returned, 0) AS amount_returned
FROM purchased p
LEFT JOIN returned r USING (customer_id);
```

- Aggregating before joining : Grouping each CTE by `customer_id` prevents row multiplication (fan-out).
- `LEFT JOIN` : Preserves customers who purchased without making returns.
- `COALESCE` : The absence of a return is recorded as a zero amount.

Form 3 : Join Truncation

SCIENCES SORBONNE UNIVERSITÉ

```sql
/* 1. Total computed through the join */
SELECT SUM(f.line_amount) AS joined_total
FROM sales.fact_sales f
JOIN sales.dim_store st USING (store_id)
WHERE f.sale_date >= DATE '2024-01-01' AND f.sale_date < DATE '2025-01-01';
-- Result: 12,400,000

/* 2. Baseline total from the fact table alone */
SELECT SUM(line_amount) AS baseline_total
FROM sales.fact_sales
WHERE sale_date >= DATE '2024-01-01' AND sale_date < DATE '2025-01-01';
-- Result: 12,900,000  -->  500,000 lost in the join
```

Validation : Baseline vs. Joined Totals

- **Detection** : Comparing the two sums reveals the fact rows whose `store_id` has no counterpart in `dim_store`.
- **No message** : An `INNER JOIN` removes unmatched records without warnings or errors.

`Unknown` Dimension Row

When the load remaps unresolved keys to an `Unknown` row, the two sums agree. The rows are kept, and the aggregate groups their amounts under a row whose attributes are all `Unknown`.

The share carried by that row is then examined in the groupings.

Join Truncation : Limitations of Sum-Based Reconciliation

Comparing two sums reveals only the difference between two values of `SUM(line_amount)`. Two errors of opposite sign leave this difference unchanged.

Underlying Row Event	Sum Check	Diagnostic Outcome
Unmatched `store_id` records	12.4 M vs 12.9 M	**Detected** : 500 k discrepancy exposed.
Simultaneous row loss AND row duplication (e.g. historized dimension)	Totals match	**Masked** : The two errors have opposite signs.
Unmatched `store_id` records whose amount is 0 or `NULL`	Totals match	**Masked** : Zero-weight rows do not impact sum.

Joining a historized dimension without temporal bounds duplicates valid rows, while missing foreign keys silently drop others.

Two Complementary Tests

- `SUM()` : the two totals are compared, and two errors of opposite sign leave the difference unchanged.
- `COUNT(*)` **and** `COUNT(DISTINCT key)` : the two counts are compared, and a loss or a duplication is revealed independently of the amounts.

State of the Data at the Observation Date

SCD Type 1 : Tracking Attribute History

SCIENCES SORBONNE UNIVERSITÉ

Mechanism : In-Place Update

Existing attributes are overwritten directly via `UPDATE`. The previous state ceases to exist, and the table keeps **one row per entity**.

State	customer_id	city
Before update	8842	Lyon
After update	8842	Paris

Slowly Changing Dimension (SCD) Type 1

Appropriate for fixing data entry errors where the previous value was factually incorrect and should not be preserved as historical reality.

Consequences for Historical Analysis

- **Loss of history** : the state of the entity at a past date can no longer be retrieved, and past transactions are described by the current value.
- **No trace** : the grain and the row count remain unchanged, and the table records nothing of the overwriting.

SCD Type 2 : Preserving History with Versioned Rows

SCIENCES SORBONNE UNIVERSITÉ

Mechanism : Row Versioning

Updates close the current row and append a new version with an active validity interval [valid_from, valid_to). The previous values are preserved.

customer_key	customer_id	city	segment	valid_from	valid_to
55231	8842	Lyon	standard	2019-01-01	**2024-06-01**
55884	8842	**Paris**	**premium**	**2024-06-01**	9999-12-31
21094	3310	Toulouse	standard	2021-03-01	9999-12-31

Temporal Validity Bounds

- **Half-open interval** : [valid_from, valid_to) cleanly partitions time without date overlap, the closing date of a row being the opening date of the next row.
- **Open upper bound** : Ongoing states use 9999-12-31 (or `NULL`). Some dimensions add a boolean column `is_current`, redundant with the interval.
- **Time-dependent state** : An attribute has meaning only relative to a specific observation date d.

Key Uniqueness

- **Natural key duplication** : `customer_id` no longer identifies a single row.
- **Surrogate key requirement** : `customer_key` identifies one state of one entity, and becomes primary key of the dimension.

Joining on Natural Key Without Date Filtering

purchased (1 row / customer)	
customer_id	amount
8842	400
3310	150
7025	220

dim_customer (SCD Type 2)			
customer_id	segment	valid_from	valid_to
8842	standard	2019-01-01	2024-06-01
8842	premium	2024-06-01	9999-12-31
3310	standard	2021-03-01	9999-12-31
7025	premium	2020-09-01	9999-12-31

```
/* The natural key joined without any condition on the validity interval */
SELECT d.segment, COUNT(*) AS n_customers
FROM purchased p JOIN customer.dim_customer d USING (customer_id)
GROUP BY d.segment;
```

Impact on Customer 8842

Customer 8842 occupies two rows of `dim_customer`, and the join returns both. Customer 8842 is counted once in `standard` and once in `premium`.

The Reasons the Error is Unnoticed

- **Partial duplication** : only the customers whose attributes changed are duplicated, and the row count increases by a few per cent.
- `DISTINCT` **does not correct the result** : `customer_id` is distinct within each segment, and the customer remains counted in two segments.

Temporal Leakage : Future State Read at Observation Date

Cohort and Computed Features

The **cohort** holds one row per pair (`customer_id`, `observation_date`).
Every feature of the pair must be computed using exclusively data available at the observation date. Using future information introduces **temporal leakage**

cohort	
customer_id	observation_date
8842	2023-01-01
8842	2025-01-01
3310	2023-01-01

dim_customer (SCD Type 2)			
customer_id	segment	valid_from	valid_to
8842	standard	2019-01-01	2024-06-01
8842	premium	2024-06-01	9999-12-31
3310	standard	2021-03-01	9999-12-31

```sql
/* The current row of dim_customer joined onto every observation date */
SELECT c.customer_id, c.observation_date, d.segment
FROM ds_martin.cohort c
LEFT JOIN customer.dim_customer d ON d.customer_id = c.customer_id
                AND d.valid_to = DATE '9999-12-31';-- one row per customer (the current value)
```

customer_id	observation_date	segment	**Status**
8842	2023-01-01	premium	**Wrong** : standard at that date, premium only from June 2024
8842	2025-01-01	premium	Correct : the state valid at that date
3310	2023-01-01	standard	Correct

Undetected by the Usual Tests

- **Row counts agree** : the three cohort rows return three rows, without duplication and without `NULL`.
- **No structural trace** : the 2023 row carries a value created in June 2024, and the checks on grain, uniqueness and missing don't detect it.

Solution : Point-in-Time Join (As-Of) Join

SCIENCES SORBONNE UNIVERSITÉ

Surrogate Key vs. Natural Key Resolution

- **Fact tables** : Join on the surrogate key (`customer_key`) which directly references the historical state at transaction time (**no date condition is needed**).
- **Cohorts** : Join on the natural key (`customer_id`) and find the active state dynamically using the `observation_date`.

```
/* Point-in-time join: Aligns entity attributes to the observation date */
SELECT c.customer_id, c.observation_date,  d.segment
FROM cohort c
LEFT JOIN customer.dim_customer d ON  d.customer_id      = c.customer_id
  AND c.observation_date >= d.valid_from
  AND c.observation_date <  d.valid_to; -- Half-open interval [valid_from,
     valid_to)
```

Implementation

- **Interval boundaries** : The condition assumes half-open intervals $[valid_from, valid_to)$
- `LEFT JOIN` **requirement** : If an entity was registered after the observation date d, no dimension record exists yet. An `INNER JOIN` would silently drop the corresponding cohort rows rows.

Validation : Grain Uniqueness and Temporal Intervals

SCIENCES SORBONNE UNIVERSITÉ

Assumption Behind the As-Of Join

As-of join assumes that the validity intervals of one customer do not overlap. Two rows valid at date d satisfy the predicate together, and the cohort row is duplicated without any message.

```sql
/* 1. Result of the as-of join: one row per (customer_id, observation_date) */
SELECT COUNT(*) AS n_rows,
       COUNT(DISTINCT (customer_id, observation_date)) AS n_keys
FROM features;
```

```sql
/* 2. Search for two rows of dim_customer valid at the same instant */
SELECT DISTINCT a.customer_id
FROM customer.dim_customer a
JOIN customer.dim_customer b
  ON  a.customer_id = b.customer_id
  AND a.valid_from  < b.valid_from        -- Report each overlap pair once
  AND a.valid_from  < b.valid_to
  AND b.valid_from  < a.valid_to
WHERE MOD(ABS(hashtext(a.customer_id::text)::bigint), 100) < 1; -- costly join,
     use 1% sample
```

Overlapping intervals are caused by the load of the dimension ⇒ question reported to the analytics team.

Fact Tables with In-Place Updates (Accumulating Snapshots)

Fact Table Type	Grain	Update Lifecycle
Transaction Fact	One event (a purchase line)	**Immutable** : inserted once, never modified.
Accumulating Snapshot	One process (an order, a claim)	**Mutable** : columns filled as the steps occur.

Mechanism : Progressive Completion of the Row

One row follows an entire process : `ordered_at`, `prepared_at`, `shipped_at`, `delivered_at`. Each column holds `NULL` until the corresponding stage of the process occurs. The load then writes the timestamp of that stage into the column.

A Leak Inherent to the Fact Table

- **The extract is written after the process ends** : an order placed on 1 January (d) and delivered on 5 January carries `delivered_at = '2024-01-05'` in an extract written in June, whereas the column held `NULL` at d.
- **The as-of join does not correct this leak** : the update occurs in the fact table, which keeps no history of its own states.

- **Recognition** : the table contains several timestamp columns, a large share of them holding `NULL`.
- **Treatment** : the column `delivered_at` gives the state at the observation date d only when its timestamp precedes d. A column still holding `NULL`, or holding a timestamp later than d, is excluded from the state at d.

Column Audit : Two Further Questions

A column raises two further questions regardless of whether its table is historized : the date at which the column was filled relative to d, and the delay separating d from the moment of prediction.

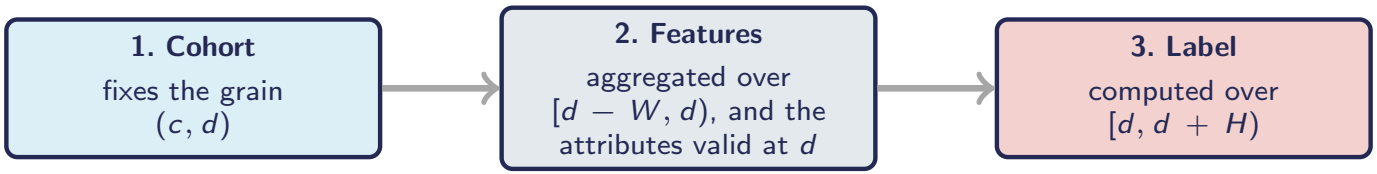
Audit Question	Failure Mode
1. When was the column filled, relative to d ?	**Closing-of-period value** : `annual_spend_2023` is written on 31 December 2023, once the year has fully elapsed. A row observed on 1 January 2023 already carries this value, although the year it summarises has not yet taken place.
2. With what delay is the column available at prediction ?	**Periodic computation** : a monthly batch fills the column for every training row. At prediction, the column holds the value of the last run, up to a month old.

Second Question

- **Training** : the extraction runs months after the observation dates, when the column is always filled.
- **Prediction** : the column holds the last run's value, aged by up to a month. Training and prediction see different ages of the same column.

Construction of the Labelled Table

Three Artefacts in Sequence



The Construction Order Is Imposed

- **Cohort first** : the cohort fixes the pairs `(customer_id, observation_date)`, and every later stage is attached to these pairs.
- **Separate stages** : the features and the label are computed in distinct `WITH` stages, each aggregated to one row per pair before the join.
- **Temporal separation** : the features read the events dated $t < d$, the label reads the events dated $t \geq d$.

Building the Cohort : Observation Dates and Eligibility

SCIENCES SORBONNE UNIVERSITÉ

The Grain of the Table

The cohort establishes the grain of the table produced : one row per customer and per observation date d.

```
/* Cohort: eligible customers, at four quarterly observation dates */
SELECT DISTINCT cu.customer_id, dt.d AS observation_date
FROM sales.fact_sales f
JOIN customer.dim_customer cu ON cu.customer_key = f.customer_key
CROSS JOIN ( VALUES (DATE '2024-01-01'), (DATE '2024-04-01'),
                    (DATE '2024-07-01'), (DATE '2024-10-01')
) AS dt(d)                        -- 4 rows, 1 column: the cross join produces the pairs
WHERE f.sale_date >= dt.d - INTERVAL '365 days'
  AND f.sale_date <  dt.d;        -- at least one purchase in the 365 days before d
```

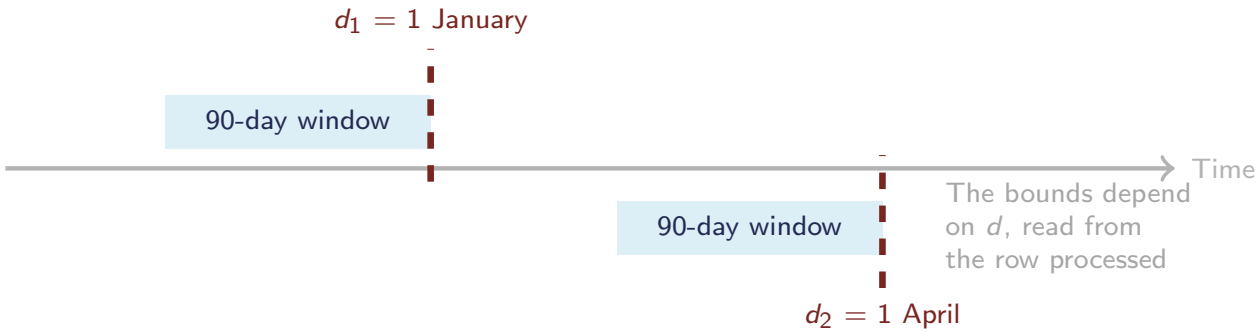
Two Decisions

- **Eligibility** : only the customers who purchased in $[d - 365 \text{ days}, d)$ enter the cohort. The model is never applied to customers inactive for longer.
- **Observation dates** : four quarterly dates spread the cohort over the year, so that the rows do not describe a single season.

Feature Windows Relative to the Observation Date

A Fixed Window and a Window Relative to d

Prior aggregates covered one period, fixed for all rows (`WHERE sale_date >= '2024-01-01'`). A feature window ends at the row's own observation date d : its bounds vary from row to row.



Placing the Conditions

- `WHERE` : one condition for the whole query, unable to vary by row.
- `JOIN` : evaluates `cohort.observation_date` per row, as in the as-of join.

Aggregation Over the Window Preceding *d*

SCIENCES SORBONNE UNIVERSITÉ

```sql
/* Facts aggregated over  [d - 90 days, d), for each pair of the cohort */
SELECT c.customer_id, c.observation_date,
    COUNT(f.sale_id)                AS n_purch_90d,  -- returns 0 when no match
    COALESCE(SUM(f.line_amount), 0) AS amount_90d -- SUM over no row is NULL
FROM ds_martin.cohort c
LEFT JOIN customer.dim_customer d ON d.customer_id = c.customer_id
                                  -- every state of the customer is matched
LEFT JOIN sales.fact_sales f ON f.customer_key = d.customer_key
  AND f.sale_date >= c.observation_date - INTERVAL '90 days'
  AND f.sale_date <  c.observation_date           -- strictly before d
GROUP BY c.customer_id, c.observation_date;
```

The Join Passes Through the Dimension

- **Two keys** : the cohort holds `customer_id`, the fact table holds `customer_key`. The dimension relates the two, and the join carries no date condition on the dimension.
- **All states are matched** : the purchases of a customer are attached to the successive surrogate keys, and their union forms the history of the customer.

Testing the Temporal Boundary of a Feature Stage

SCIENCES SORBONNE UNIVERSITÉ

`COUNT` and `SUM` remove the dates of the events : after the `GROUP BY`, the result holds no `sale_date`. To verify temporal integrity, test the unaggregated intermediate join against the observation date.

```
WITH before_agg AS (/* The feature stage, minus the GROUP BY: sale_date is kept */
  SELECT c.customer_id, c.observation_date, f.sale_date
  FROM ds_martin.cohort c
  LEFT JOIN customer.dim_customer d ON d.customer_id = c.customer_id
  LEFT JOIN sales.fact_sales f ON f.customer_key = d.customer_key
    AND f.sale_date >= c.observation_date - INTERVAL '90 days'
    AND f.sale_date <  c.observation_date
)
SELECT COUNT(*) AS n_violations FROM before_agg
WHERE sale_date >= observation_date;   -- must return 0
```

Each Row Compared With Its Own Observation Date

- A constant in place of `observation_date` tests every row against the same bound : the violations of the windows before the constant remain undetected.
- A non-zero result : `COUNT(*)` replaced by `*` returns the rows concerned, and the dates carried by one of those rows show which join condition is wrong.

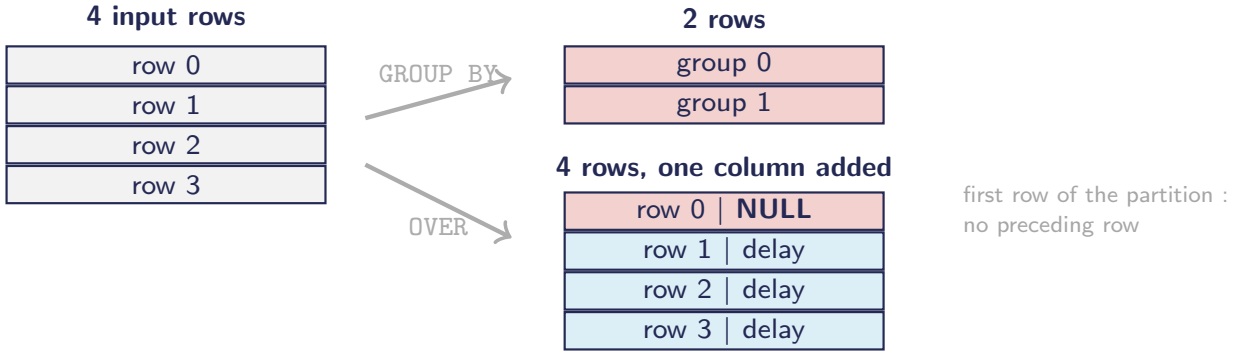
The Same Test on the Label Stage

The inequality is reversed : a row carrying `sale_date < observation_date` indicates an event of the feature window counted in the label.

Window Functions : Preserving the Grain of the Rows

The Limitation of `GROUP BY`

`GROUP BY` merges the rows and changes the grain of the result.
A measure computed between two rows, such as the delay between two consecutive purchases, requires every event to remain visible while the preceding event is read.




```sql
/* The delay between two consecutive purchases, added to each row */
sale_date - LAG(sale_date) OVER (PARTITION BY customer_key ORDER BY sale_date)
--          ^ function       ^ the group                   ^ the order within the group
```

- `PARTITION BY` defines the group, `ORDER BY` the order in which the rows are read.
- **Evaluation** : at the `SELECT` stage. A `WHERE` clause cannot filter the column produced.
- **Grain** : the column represents the grain of the event. A further aggregation is needed to get the grain of the cohort.
- **First row of a partition** : `LAG` returns `NULL` (0 means same day, not an absence). `LEAD` returns the following row.

Construction of the Label

Definition

Churned (1) at date d if no purchase occurs in $[d, d + 90 \text{ days})$, Active (0) otherwise.

```sql
/* The events of the window [d, d + 90 days), counted for each pair of the cohort */
SELECT c.customer_id, c.observation_date,
       CASE WHEN COUNT(f.sale_id) = 0 THEN 1 ELSE 0 END AS has_churned
FROM ds_martin.cohort c
LEFT JOIN customer.dim_customer d ON d.customer_id = c.customer_id
LEFT JOIN sales.fact_sales f
  ON  f.customer_key = d.customer_key
  AND f.sale_date   >= c.observation_date                        -- lower bound: d included
  AND f.sale_date   <  c.observation_date + INTERVAL '90 days'  -- upper bound: excluded
GROUP BY c.customer_id, c.observation_date;
```

No Boundary Overlap

A purchase dated exactly d falls in the label window, not the feature window : the same half-open convention governs both stages.

A Separate Stage

Compute the label table in an independent stage/CTE before joining back to the cohort spine to prevent Cartesian fan-out with feature tables.

Assembling the Labelled Table

SCIENCES SORBONNE UNIVERSITÉ

```sql
WITH cohort   AS ( ... ), -- one row per (customer_id, observation_date)
     features AS ( ... ), -- aggregates over the window strictly before d
     label    AS ( ... ), -- churn outcome over the window [d, d + 90 days)
     attrs    AS ( ... )  -- as-of join on dim_customer at date d
SELECT c.customer_id, c.observation_date,
       f.n_purch_90d, f.amount_365d,
       a.segment,
       l.has_churned
FROM cohort c
LEFT JOIN features f USING (customer_id, observation_date)
LEFT JOIN label    l USING (customer_id, observation_date)
LEFT JOIN attrs    a USING (customer_id, observation_date)
WHERE c.observation_date < :history_end - INTERVAL '90 days';
```

Assembly by `LEFT JOIN`

- **Grain preserved** : every stage is aggregated to one row per pair before the join, and the result holds as many rows as the cohort.
- **Fan-out** : two fact tables attached to the cohort within one query multiply the rows, and the aggregation performed inside each stage prevents it.

Observation Dates Excluded by the Final `WHERE`

- **Incomplete label window** : an observation date less than 90 days from the end of the history covers a period the warehouse has not loaded, and the label records the missing period instead of the behaviour of the customer.
- `:history_end` **is a parameter** : `SELECT MAX(sale_date)` returns one value and reads every partition of `fact_sales` to obtain it.

Validating the Labelled Table

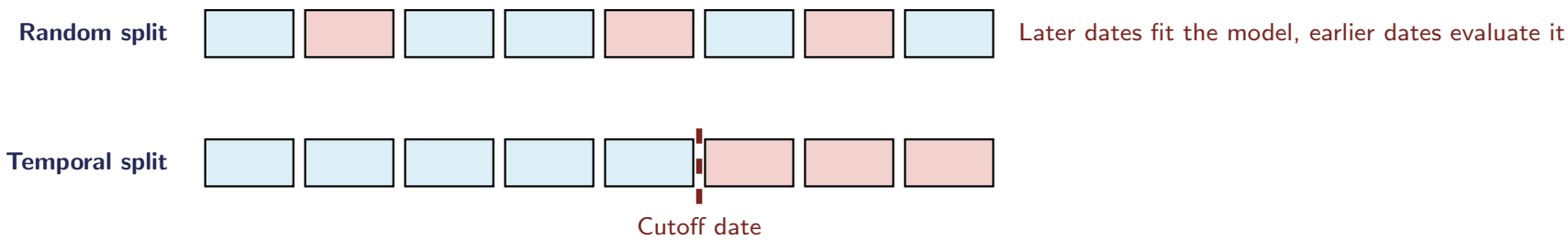
Check	Cause of a Failure
Row count matches the cohort	Every join is a `LEFT JOIN` from the cohort, which preserves every pair. An excess indicates a stage holding several rows for one pair (`customer_id, observation_date`).
No unintended `NULL`	A `LEFT JOIN` without a matching event returns `NULL` unless a `COALESCE` replaces it with zero. The query knows this `NULL` denotes zero. The modelling library reading the exported table does not, and may impute it as missing data.
Proportion of positive labels	A rate of 0 % or 100 % indicates a boundary error in the label window. A rate far from the figure published by the company indicates that the coded definition and the official definition differ.
Temporal boundary	A row carrying `sale_date >= observation_date` in a feature stage indicates temporal leakage.

The four checks are written in SQL and run before the table is read into Python.

Temporal Train-Test Split

Criterion of the Split

A deployed model predicts at dates posterior to every observation used to fit it.
A random split places part of the evaluation dates before the fitting dates. The split is therefore written as a condition on `observation_date`.



Gap Left at the Boundary

- An observation dated 1 June carries a label computed on the purchases up to 30 August. A test set opening on 1 July reads the same purchases.
- The gap left empty between the last training date and the first test date equals the label window, 90 days.

Constructing the Scoring Table

Same Features, No Label

A fitted model is applied to customers whose outcome has not occurred.
The extraction reuses the query that produced the labelled table, with three modifications.

```
/* Scoring table: the features evaluated at the date :d */
WITH cohort   AS ( ... WHERE d = :d ), -- 1. one date, passed as a parameter
     features AS ( ... ),              --    the same text, character for character
     attrs    AS ( ... )               --    the same text
                                       -- 2. the label stage is removed
SELECT c.customer_id, c.observation_date,
       f.n_purch_90d, f.amount_365d,
       a.segment
FROM cohort c
LEFT JOIN features f USING (customer_id, observation_date)
LEFT JOIN attrs    a USING (customer_id, observation_date);
                                       -- 3. the final WHERE is removed
```

Three Modifications, Two Reused Stages

- **Single observation date** : Several dates supplied the examples used for fitting. A prediction bears on the current state, so the cohort holds the single date `:d`.
- **Label stage removed** : the window $[d, d + 90 \text{ days})$ lies in the future.
- **Final `WHERE` removed** : the clause excluded the dates whose label window was incomplete.
- **Identical feature logic** : The reuse of the same stages keeps the definitions of the training extraction and of the scoring extraction identical.

Reproducibility of an Extraction

Modification of Data Already Loaded

Mechanism	Example
Source data correction	A point-of-sale fault affecting the transactions of March is corrected at the source in April, and the next load overwrites the already loaded rows of March with the corrected values.
Definition modified	Returns are subtracted from revenue, and the analytics team re-executes the transformations over the whole history.
Late-arriving events	A transaction of 30 March reaches the warehouse in April, and the load writes it into March.

Consequence for a Project

- **Two runs of the extraction** : a feature is added to the extraction between the two runs, and the two models obtain different scores.
- **Two unresolved causes** : the added feature may improve the prediction, or the data of the period may have changed between the two runs. Either cause produces the same difference between the two scores.
- **The table of the first run is kept** : the difference between the two scores measures the feature alone.
- **The extraction is re-run against the state of the warehouse at the first run** : the difference measures the feature together with every modification made to the extraction.

Recording Metadata Alongside the Extraction

```
# 1. The parameterised SQL is read from a version-controlled file
sql = open("extract_churn.sql").read()
# bounds of the observation date range
params = {"d0": DATE_START, "d1": DATE_END}
df = pd.read_sql(text(sql), engine, params=params)

# 2. A manifest is written beside the dataset
manifest = {
    "executed_at": datetime.datetime.now().isoformat(),
    "n_rows":      len(df),
    "sum_amount":  float(df["amount_365d"].sum()),
    "query_sha":   hashlib.sha256(sql.encode()).hexdigest()
}
json.dump(manifest, open("churn_2024.manifest.json", "w"))
```

Comparison of Two Manifests

- **Code provenance** (`query_sha`) : two different hashes establish that `extract_churn.sql` was modified between the two runs.
- **Data state** (`n_rows`, `sum_amount`) : values differing under an identical hash establish that the data read changed between the two runs.

Version Control of the Extraction Query

`extract_churn.sql` holds the extraction query used to compute the labelled table.

When No Manifest Was Kept

Inspecting Transformation Changes (Code)

The transformations that produce the warehouse tables live in a version-controlled repository, dbt being a common tool. The version history gives the modifications applied to a column definition between two runs.

Past States of the Tables (Data)

Engines supporting **time travel** (Snowflake, BigQuery, Delta Lake, Iceberg) can valuate a query as the tables stood at a past instant. The extraction re-run at the instant of the first run reads the same data.

Practical Limit

- **Short retention** : the retention period is counted in days, one by default on Snowflake and seven on BigQuery.
- **Duration of a project** : a project extends over weeks, and the past state sought no longer exists when the comparison becomes necessary.
- **These two means are recourses. The manifest written at the first run holds for the duration of the project.**

Synthesis : The Warehouse and the Errors It Produces

Properties of the Warehouse

The load computes the tables of the warehouse, and the analytics team fixes the content of every column.
The account grants read access on the shared schemas and write access on one personal schema.

Error	Form It Takes
Meaning of a row	A computed figure differs from a published figure, and the two definitions differ.
Additivity	A stock summed along time returns a multiple of the stock.
Fan-out	A join between two fact tables multiplies the amounts of each table.
Join truncation	An `INNER JOIN` removes the customers without events.
Temporal leakage	A feature reads events posterior to the observation date.

Property Common to the Five Errors

The engine raises no error, and the figure returned remains plausible. A dedicated check detects each of the five errors.

Synthesis : The Construction of the Labelled Table

Four Stages

- **Cohort** : one row per customer and per observation date, which fixes the grain.
- **Features** : the events dated $t < d$, aggregated.
- **As-of join** : the dimension row valid at d.
- **Label** : the events dated $t \geq d$, counted.

Each stage returns one row per pair, and the assembly attaches the stages to the cohort by `LEFT JOIN`.

Verifications

- The row count of the result equals the row count of the cohort.
- Every `NULL` produced by a `LEFT JOIN` on an amount receives a `COALESCE`.
- No row of a feature stage carries an event dated d or later.

Scoring Table and the Manifest

The scoring extraction reuses the same stages, with one observation date and without the label stage.
A reload modifies a period already loaded, and the manifest written at the first run establishes whether two runs read the same data.